

AIHealthPortal - an agentic AI driven platform to match patients with doctors, using a Large Language Model and supporting optimal resource utilization for a hospital system

Project Advisor: Gopinath Vinodh

Team Members:

Lakshmi Sriya Amperayani

Harshavardhan Reddy Gadila

Jayasurya Murali

Veera Venkata Vijaya Subhash Polisetti

Slide 2: Introduction

Medical artificial intelligence has shifted healthcare operations toward using large language models (LLMs) and multi-agent workflows.

Foundational work in retrieval-augmented generation (RAG) and vector databases has transformed how medical knowledge is synthesized.

Modern AI allows for dynamic, context-aware decision-making within clinical settings.

Slide 3: Problem Statement - Hospital Inefficiencies

Hospital scheduling and triage workflows are primarily availability-driven.

Simplistic, rigid rules frequently ignore critical contextual factors like a doctor's specific clinical expertise and historical case outcomes.

This results in delayed or suboptimal care for patients, while specialized doctors remain under-utilized.

Slide 4: Proposed Solution – AIHealthPortal

AIHealthPortal is an orchestrated suite of specialized AI components designed to enhance operational efficiency and clinical accuracy.

The system utilizes modern Large Language Models (LLMs), RAG, and multi-agent orchestration frameworks.

It introduces three core agents to manage patients, emergencies, and doctor support.

Slide 5: Core AI Agents

Patient Matchmaker Agent: Analyzes symptoms and uses semantic vector search to recommend the most suitable specialist based on clinical experience.

Emergency Handler Agent: Evaluates symptom urgency through RAG-augmented reasoning, bypassing standard queues for critical cases.

Doctor Support Assistant Agent: Retrieves and summarizes similar past cases and evidence-based guidelines from the vector database for physicians.

Slide 6: Literature & State of the Art Summary

Retrieval-Augmented Generation (RAG) systems combine generative reasoning with external data retrieval to overcome model hallucinations.

Vector databases like ChromaDB provide efficient semantic search across unstructured hospital records.

AIHealthPortal bridges the gap between academic frameworks and real-world industrial deployments using open-source, on-premise AI models.

Slide 7: High-Level Project Architecture

The system employs a client-server, modular, and layered architecture divided into three primary functional subsystems.

Frontend Layer: Role-specific client interfaces built for patients, doctors, nurses, and admins.

Backend Layer: Middle-tier managing API routing, secure authentication, and LangChain orchestration.

Data & AI Infrastructure: Manages semantic reasoning through vector storage and structured data through a relational database.

Slide 8: Technology Stack - Client & Middle Tier

Frontend UI: Built using a JavaScript library (React) for building component-based, responsive web applications.

Middle-Tier Backend: Powered by Python and FastAPI to natively support asynchronous programming.

Orchestration: LangChain acts as the connective link managing tool interactions and RAG pipelines for the multi-agent system.

Slide 9: Technology Stack - Data Tier & AI

Vector Database: ChromaDB serves as the primary engine for the RAG pipeline, storing 1024-dimensional embeddings of doctor expertise and patient cases.

Relational Database: Google Cloud SQL DB maintains strict atomic state changes and tracks real-time scheduling data to prevent double-booking.

AI Models: Mistral AI provides both the embedding models for vector conversion and chat models for generative reasoning.

Slide 10: Client Subsystem Design

Patient Portal: Allows patients to input symptoms and demographic info, rendering top-k matching doctors with AI-generated explanations.

Doctor Portal: Integrates with the Doctor Support Assistant Agent to present summarized historical cases and treatment outcomes.

Interactive Booking: Triggers a seamless booking interface that updates UI state without a full page refresh.

Slide 11: Middle-Tier Orchestration Design

Governed by LangChain, the middle-tier orchestrates context-sharing between Mistral LLMs and databases.

The `find_best_doctor` matching engine retrieves candidates from ChromaDB and cross-references availability against Google Cloud SQL DB.

The Mistral Chat Service generates explanatory text for the final JSON payload sent to the patient.

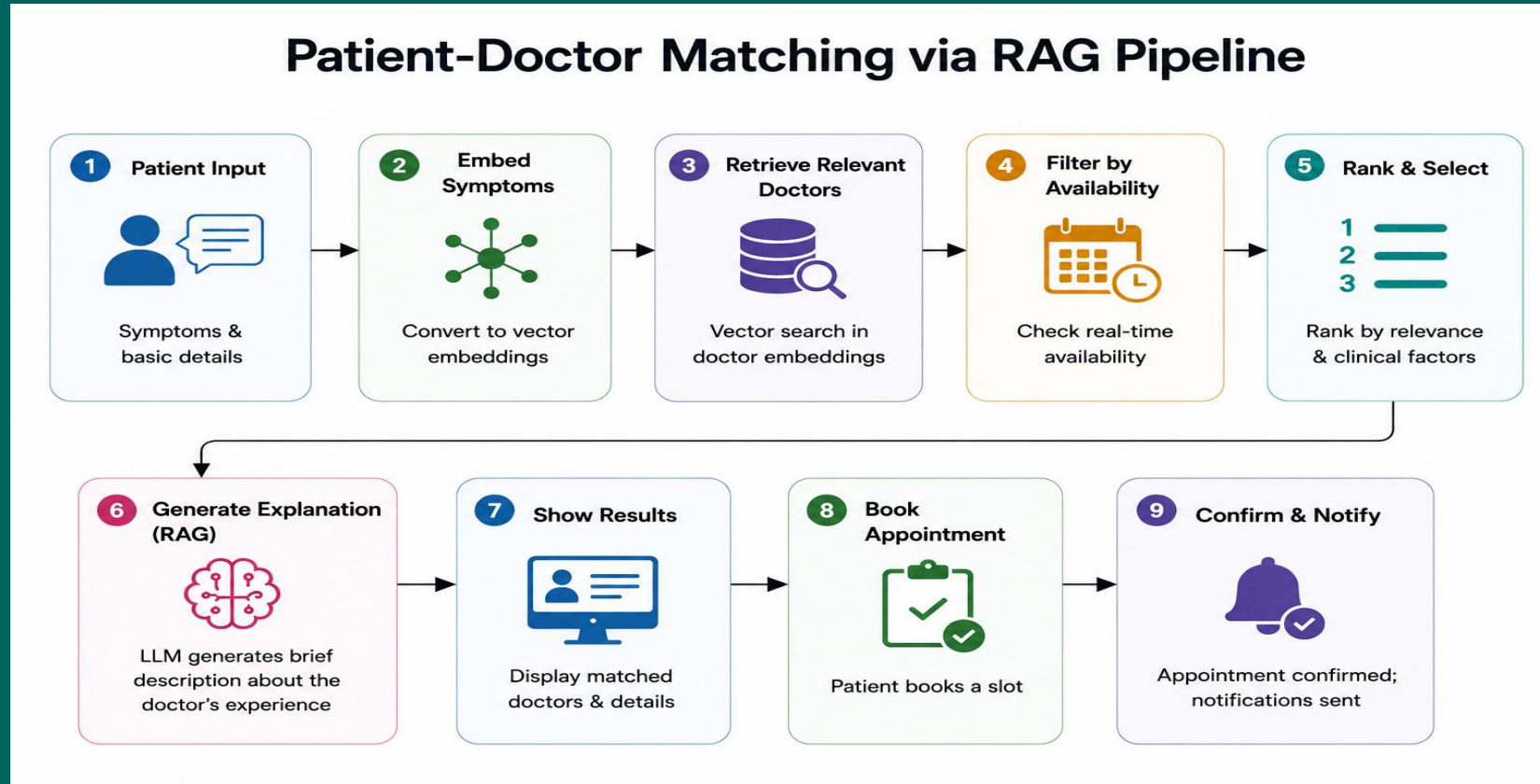
Slide 12: Data-Tier Schema Design

ChromaDB calculates cosine distance to rank doctors mathematically based on clinical relevance.

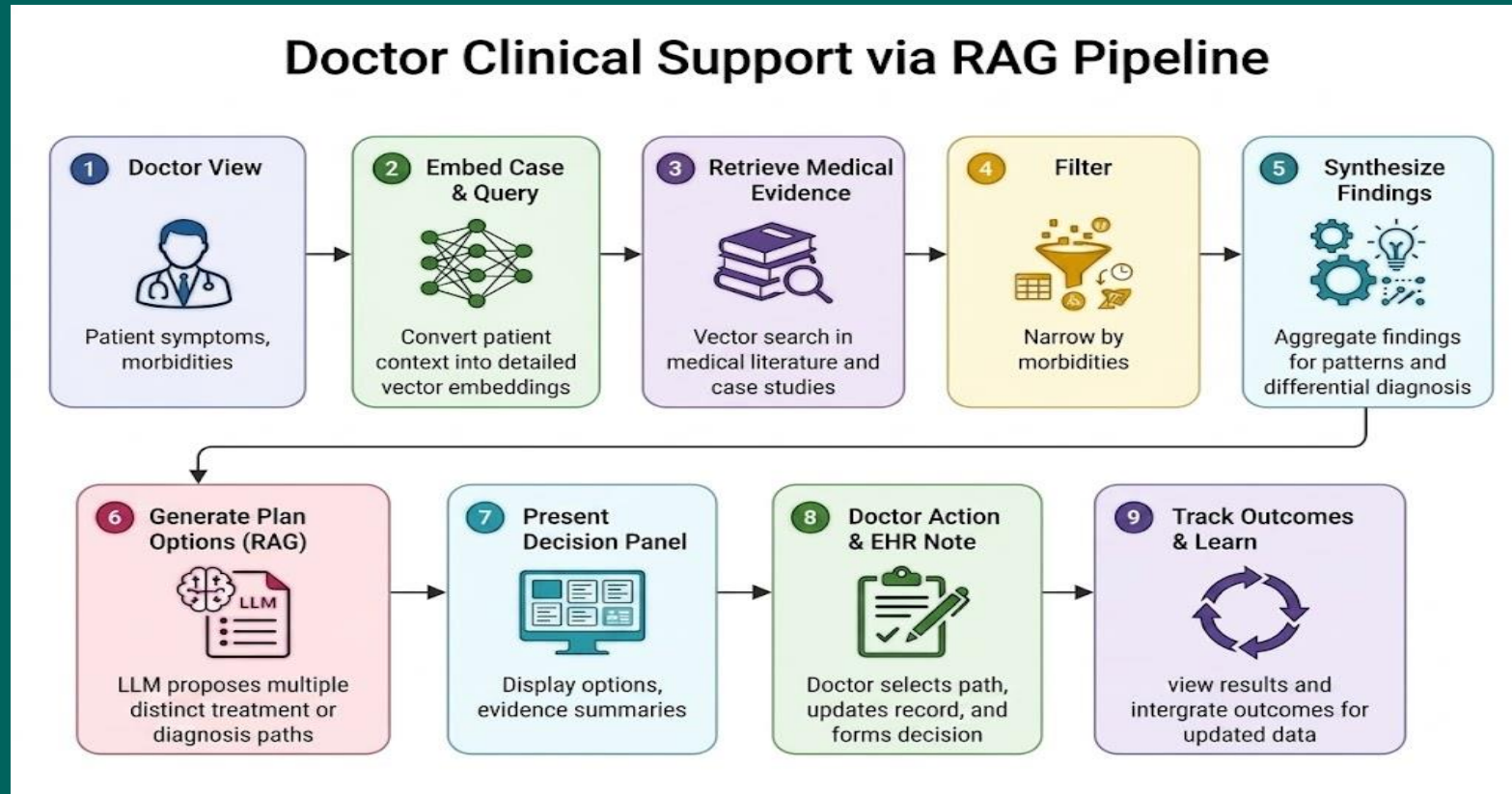
The Google Cloud SQL database maintains doctors, users, appointments, and app_users tables.

A unique constraint is enforced on the doctor_id, appointment_date, and slot_start_time combination to prevent double-booking.

Slide 13: Workflow 1 - Patient-Doctor Matching



Slide 14: Workflow 2 - Doctor Clinical Support



Slide 15: Emergency Handling –Vital Signs Monitoring

The screenshot shows a web browser with multiple tabs. The active tab is titled 'frontend' and the address bar shows '10.0.0.35:5173/emergency-vitals'. The page content is as follows:

Emergency vitals monitoring

Live pulse, oxygen, blood pressure, respiration, and temperature.

[Home](#) [Logout](#)

Live vitals

Pulse, SpO₂, blood pressure, respiratory rate, and temperature — updated in real time. Simulated feed for training scenarios; not for clinical diagnosis.

● Live stream

Heart rate

72.8 bpm

SpO₂

97.4%

Blood pressure

116/76

Resp. rate

14.1 /min

Temperature

98.6 °F

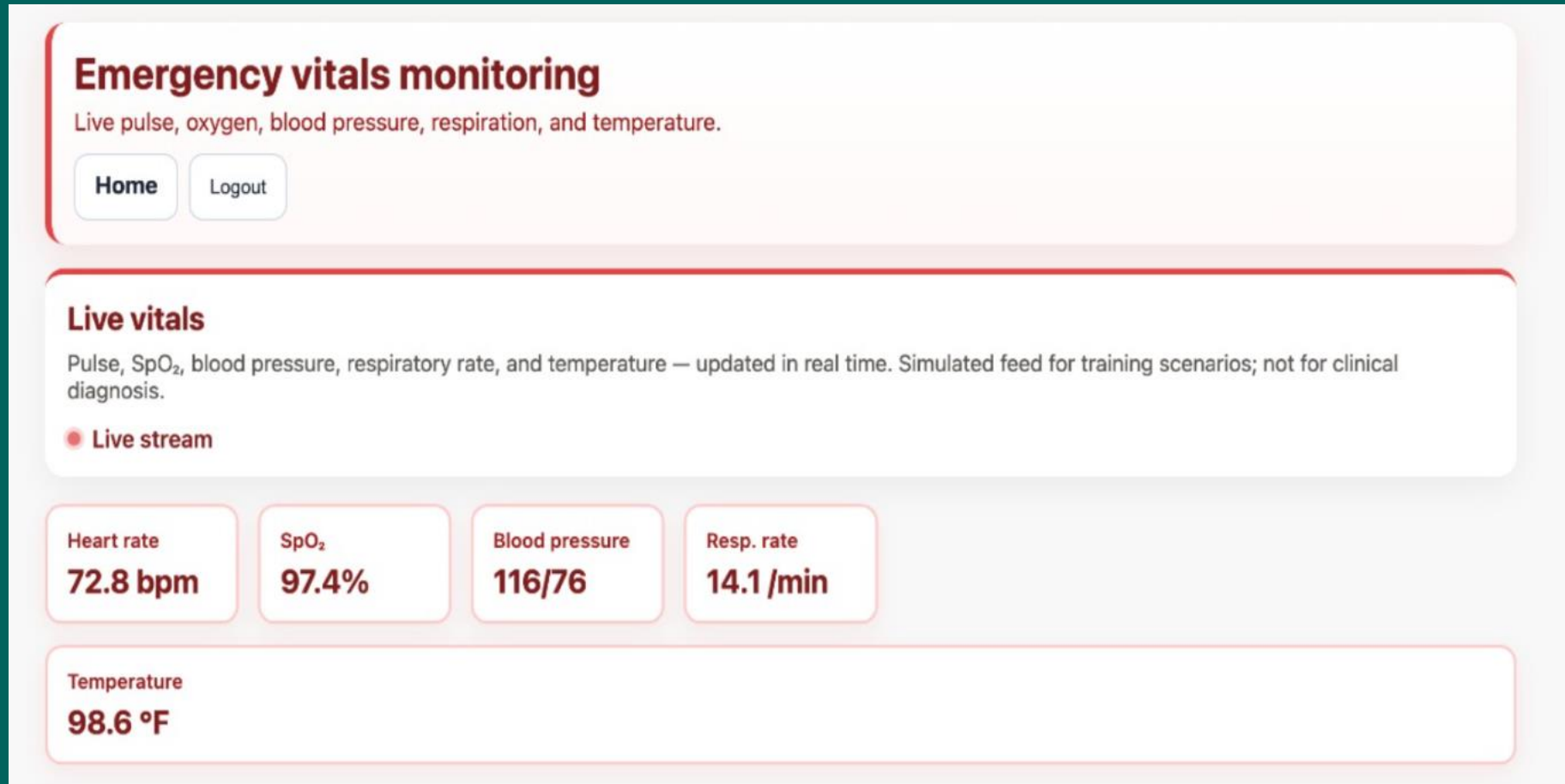
Slide 16: Implementation & Agile Process

The project utilized an Agile Scrum-based methodology across four dedicated sprints over twelve weeks.

Environments were containerized via Docker to ensure consistency and deployed to Google Compute Engine (GCP) for scalability.

A background sync service continuously updates doctor records from ChromaDB into the Google Cloud SQL tracking table.

Slide 17: AIHealthPortal Website Sample Images



Slide 18: AIHealthPortal Website Sample Images

The screenshot displays a web browser window with multiple tabs. The active tab shows a web application titled "Doctor-Patient Matching System". The browser's address bar indicates a "Not Secure" connection to the IP address "10.0.0.35:5173".

The application interface includes a header with the title "Doctor-Patient Matching System" and a sub-header "Enter your symptoms below to find the best matching doctor for your needs." Below this, there are three buttons: "Find Doctor" (highlighted in red), "My Appointments", and "Logout".

The main section is titled "Describe your symptoms:" and contains a text input field with the text "Having sever stomache pain". Below the text field are three input fields: "Age" (containing "27"), "Gender" (a dropdown menu showing "Female"), and "ZIP" (containing "95112"). A blue error message box below the ZIP field states "Location permission denied. Enter ZIP manually." A large red button labeled "Find Doctors" is positioned below the input fields.

The bottom section is titled "Matched doctors" and lists two doctors:

- Dr. Jonathan Erber, MD**
Gastroenterologist
[View location on Google Maps](#)
- Dr. Faisal Jamal, MD**
Nephrologist
[View location on Google Maps](#)

Slide 19: AIHealthPortal Website Sample Images

Clinical Decision Matching System

Search similar patient cases to support clinical decision-making.

Enter clinical presentation:

knee surgery
aged person having CABG done already

Number of similar cases

5

Show detailed case information

☒

Find Similar Cases

Slide 20: Conclusion & Future Recommendations

Conclusion:

Vector-based semantic search and RAG workflows vastly outperform traditional availability-driven manual scheduling of appointments.

Future Recommendations:

Open-source models, built-from-scratch models, or private-cloud-hosted models within a designated region ensure sensitive patient data remains strictly compliant with healthcare data privacy regulations.

The background is a solid teal color. In the bottom-left corner, there is a large, dark teal abstract shape that resembles a hill or a wave. Overlapping this and extending towards the top-right is a lighter teal, semi-transparent shape that looks like a large, curved arrow or a stylized 'S' shape.

Thank You